comm_scripts: user guide

What the code does and how to run each part

skipper-CCD module testing

version of 2026-10-25 (branch `clustering-methods-2026-10-25`)

Abstract

`comm_scripts` takes raw skipper-CCD images to calibrated results with panaSKImg (WADERs). It writes the panaSKImg configuration itself and measures gain, noise and dark current per CCD with errors. It can correct the gain's dependence on pixel charge, run the Fe-55 composite analysis, and cluster with A_GMX, a clusterer that writes files in the WADERs format. The repository also contains standalone utilities, the research code of the clustering study, and the scripts behind the analysis notes. This guide lists every part, shows how to run it, and says which inputs it needs and which outputs it writes. Section 2 covers the common tasks in a few lines each. The physics behind the pipeline steps is in `docs/PIPELINE_GUIDE.md`, and the A_GMX algorithm is described in `docs/aGMX/aGMX_clustering.tex`.

Contents

1 Overview **3**

1.1 What is in the repository . . . 3

1.2 Requirements and set-up . . . 3

1.3 Conventions used throughout . . . 3

2 Common tasks in a few lines **4**

3 The pipeline: `run_WADERs` **4**

3.1 What it does, step by step . . . 4

3.2 Options . . . 5

3.3 Output folder . . . 6

4 Gain versus charge: `--gain-correction` and `gain_vs_charge.py` **6**

4.1 Inside `run_WADERs` . . . 7

4.2 On its own . . . 7

5 Fe-55 composite analysis: `--composite` **7**

6 A_GMX clustering **8**

6.1 Three ways to run it . . . 8

6.2 File format . . . 8

6.3 From Python . . . 9

7 The pipeline modules on their own **9**

7.1 `autoconfig_panaSKImg.py`: a panaSKImg JSON from a raw image . . . 9

7.2 `gain_dc_split.py`: gain from all peaks, dark current from two 10
7.3 `fit_quality.py`: how good are the dark-current fits 10
7.4 `gain_vs_charge.py` and `agmx_root.py` 10

8 Utilities **10**

9 Research code: `clustering_tests/` **11**
9.1 Methods 11
9.2 Test and validation scripts 12
9.3 Documentation builders 12

10 Analysis notes and reports **12**
10.1 Am-241 lines, 10–30 keV (`docs/am241_note/`) 12
10.2 Gain correction 13
10.3 Clustering 13
10.4 Hand-over notes 13

11 Tests and changing the code safely **13**

12 Troubleshooting **14**

1 Overview

1.1 What is in the repository

folder	kind	contents
`bin/`	commands	`run_WADERs` (alias `runWADERs`): the full pipeline; `run_AGMX`: A_GMX clusters for a finished run
`src/`	pipeline modules	`autoconfig_panaSKImg.py`, `gain_dc_split.py`, `fit_quality.py`, `gain_vs_charge.py`, `agmx_root.py` (all also runnable on their own)
`utils/`	standalone helpers	`compare_runs.py`, `split_clustersRec_by_ext.py`, `compare_dcfit_vs_skips.py`, `skip_noise_L1.py`
`clustering_tests/`	research code	the clustering methods (A_GMX, GMX, MHC, MFD+G, hybrid, ...) and the scripts that tested them
`tests/`	unit tests	simulated inputs, no data needed; run in under a minute
`docs/`	guides, notes, reports	`PIPELINE_GUIDE.md`, the A_GMX document, the Am-241 note, the gain-correction note and reports, this guide, hand-over notes

Keep `bin/` and `src/` next to each other: `run_WADERs` and `run_AGMX` load their modules from `../src`. `src/agmx_root.py` also imports the clustering code from `../clustering_tests`.

1.2 Requirements and set-up

- **Python environment**: numpy, scipy, astropy, matplotlib, PyROOT and pysimdamicm (panaSKImg). On this machine that is the conda environment `damic`.
- **Automatic environment**: `run_WADERs` and `run_AGMX` can be started from any python. If that python lacks ROOT or pysimdamicm, they look for a conda/mamba environment that has panaSKImg and re-run themselves there with `conda run`. Nothing is left activated afterwards.
- **Choosing the environment**: `--env NAME` or `$WADERS_ENV`. To choose the panaSKImg script, use `--panaskimg PATH` or `$PANASKIMG`.
- **Optional**: ROOT's `root` executable, for `--composite` (it runs the `MakeComposite_Fe55_Clusters_Extensions.cc` macro). `tectonic` or pdflatex, to build the LaTeX documents.
- **Calling by name**: to run the commands from anywhere, add `export PATH="/path/to/comm_scripts/bin:$PATH"` to `~/.zshrc`.

Environment variables

`WADERS_ENV`	conda environment to use (`run_WADERs`, `run_AGMX`)
`PANASKIMG`	panaSKImg script
`MAKECOMPOSITE_MACRO`	`MakeComposite_Fe55_Clusters_Extensions.cc` used by `--composite`
`AM241_RUN_DIR`	data folder of the Am-241 note scripts (Section 10)

1.3 Conventions used throughout

- **Inputs**: a *raw image* is a skip-averaged FITS image (`.fz`, `.fits`, `.fits.bz2`), with one HDU per amplifier, i.e. per CCD (`EXT1`–`EXT4`). A *list* is a `.txt` file with one image path per line. Relative paths are relative to the `.txt`; empty lines and `#` lines are ignored.
- **Output folder**: every run writes `WADERs_analysis_<run name>/`. The run name is the image name, or the `.txt` name for a list.

- **Cluster files**: `panaSKImg_clustersRec_<image>_ext<N>.root`, one per CCD, with trees `clustersRec` (one entry per image, one vector element per cluster), `info` and `process_config`. Energies are in keV with 3.74 eV per electron.
- **Modules**: the ACM board id appears in the image name (`..._106_20260330_123052_21`: ACM 106). Images with the same ACM share the CCDs and are grouped, e.g. for one gain curve per module.
- **Fe-55**: results are quoted for front-side clusters ($\sigma_{xy} < 1$ px) unless stated otherwise.

2 Common tasks in a few lines

I want to ...	run
analyse one image	`run_WADERs image.fz`
analyse a list, 8 at a time, elsewhere	`run_WADERs images.txt -o ~/analysis -j 8`
only gain, noise, dark current (fast)	`run_WADERs images.txt --no-clusters`
gain-corrected cluster energies	`run_WADERs images.txt --gain-correction`
compare per-image and per-module $g(n)$	`run_WADERs images.txt --gain-correction both`
Fe-55 composite plots per module	`run_WADERs images.txt --gain-correction --composite --module 109=DM02`
cluster with A_GMX in the pipeline	`run_WADERs images.txt --gain-correction --clusterer agmx --composite`
WADERs and A_GMX side by side	`run_WADERs images.txt --gain-correction --clusterer both --composite`
A_GMX for a run already done	`run_AGMX WADERs_analysis_<run>`
cluster thresholds in electrons	`run_WADERs images.txt --autoconfig-args "--cluster-threshold-e auto"`
check a code change changes nothing	run old and new code with `-o old` / `-o new`, then `python utils/compare_runs.py old new`
one JSON for panaSKImg, by hand	`python src/autoconfig_panaSKImg.py image.fz -o image.json`
split an `_ext1234.root` file per CCD	`python utils/split_clustersRec_by_ext.py f_ext1234.root -o per_ext/`
noise vs number of skips (level-1 data)	`python utils/skip_noise_L1.py img_..._L1_....RJ45ADCDATA0 -o out/`
run the unit tests	`python -m unittest discover -s tests -v` (repository root, `damic` python)

3 The pipeline: `run_WADERs`

```
run_WADERs <image.fz | image.fits | image.fits.bz2 | list.txt> [-o PARENT] [-j N] [options]
run_WADERs -h          # the options
run_WADERs -h -v       # the full guide: output folders, what each step does, all autoconfig
    settings
```

3.1 What it does, step by step

Each step is one function in `bin/run_WADERs`; `main()` calls them in this order.

step	what happens
check_request	inputs exist, the output folder is free (else --overwrite), the options are compatible
ensure_environment	re-runs inside the conda environment with panaSKImg if needed
step1_configure	per CCD, gain_dc_split.py --per-ext writes two configurations, both measured from the raw pixels: a *dark-current* JSON, which fits only the Poisson-like $0\,e^-/1\,e^-$ peaks, and a *gain* JSON, which fits all peaks. Both are fitted with panaSKImg. The final JSON holds the all-peaks gain if that is more precise (errors scaled by $\sqrt{\chi^2/\mathrm{ndf}}$) and agrees within 3σ. Flooded or unresolved CCDs are flagged.
step2_run_panaskimg	panaSKImg once per CCD (--ext N), in parallel (-j)
step3_collect_root_files	cluster files go to root/, dark-current fits to root/dc_fits/, and the file list to root/root_files.txt
step3b_gain_correction	with --gain-correction: measure $g(n)$ and write corrected cluster files (Section 4)
step3d_agmx	with --clusterer agmx|both: A_GMX cluster files in the same format (Section 6)
step3c_composite	with --composite: the MakeComposite macro per image type and module (Section 5)
step4_summary	fit_quality.csv and summary.txt: gain, noise and dark current per CCD with errors, which gain was used and why, and the results of the optional steps

3.2 Options

option	meaning
general	
-o DIR, --parent DIR	folder in which WADERs_analysis_<run> is created (default: current folder)
-j N, --jobs N	panaSKImg runs at the same time (default 4, or fewer CPUs)
--overwrite	replace an existing output folder
--all-ext	also run CCDs flagged as flooded/unresolved (slow; fits not reliable)
--no-clusters	noise, gain, dark current only: no ClusterFinder (much faster)
--tune	per CCD, choose the pedestal method and fit window with panaSKImg by the smallest total fit error (∼1 min more per image)
--keep-work	keep the intermediate fit runs in work/
--autoconfig-args "..."	settings for the JSON generation, in one quoted string (Table 4)
--env NAME, --panaskimg PATH	environment and panaSKImg script
gain versus charge (Section 4)	
--gain-correction [MODE]	pooled (default: one curve per module), per-image, both
--gain-model	interpolated (default), broken, linear
--gain-combine	coherent (default), incoherent
--gain-anchor	comb (default), fit
clustering (Section 6)	
--clusterer	waders (default: panaSKImg's ClusterFinder only), agmx (also A_GMX files, used by --composite), both (--composite on both)
composite (Section 5)	
--composite	run MakeComposite on the cluster files
--composite-macro PATH	the macro (default $MAKECOMPOSITE_MACRO, else the one in ~/Documents/LSM_Module_Testing_Analysis/src/)
--module ACM=NAME	module name of an ACM, e.g. --module 109=DM02 (repeatable)

option	meaning
`--image-number N`	image type for the macro (default: `Image_<N>` in the name)

3.3 Output folder

path (in the run folder)	contents
`*.pdf`, `*_EXT<n>_waders.fits.bz2`	panaSKImg plots; the calibrated image of each CCD (electrons)
`summary.txt`, `fit_quality.csv`	gain, noise, dark current per CCD with scaled errors and the dark-current bias
`root/`	panaSKImg cluster files, one per CCD, and `root_files.txt`
`root/dc_fits/`	dark-current fits (histogram + function) as ROOT
`json/<image>/final/`	the JSONs that were run (one per CCD)
`json/<image>/gain/`, `dc/`	intermediate JSONs; `<image>_gain_dc_summary.txt` compares the fits
`logs/`	panaSKImg, autoconfig and MakeComposite logs
`gain_vs_charge/`	$g(n)$ curves (JSON) and their plots
`root_gaincorr/`, `root_gaincorr_per_image/`	gain-corrected cluster files
`root_agmx/`, `root_agmx_gaincorr*/`	A_GMX cluster files
`composite*/`	MakeComposite plots and the file lists given to it

Reading `summary.txt`

For each CCD the summary gives the χ^2/ndf of the dark-current fit, and the gain, noise and dark current with errors scaled by $\sqrt{\chi^2/\mathrm{ndf}}$. `*` marks a gain held at the all-peaks fit. The *DC bias* is the fitted dark current minus the one from the $0\,\mathrm{e}^-/1\,\mathrm{e}^-$ pixel counts; it should be close to 0. Below the table come the reason for each gain choice, the skipped CCDs, and the results of the optional steps.

4 Gain versus charge: `--gain-correction` and `gain_vs_charge.py`

The gain falls slowly with the charge in a pixel ($\sim$3 % at $1000\,\mathrm{e}^-$). Converting every pixel with the low-charge gain therefore puts keV clusters low. `gain_vs_charge.py` measures $g(n)$ from the spacing of the electron comb at high charge, in the images themselves, using clean rows. It then converts each pixel's ADU to electrons with that curve.

4.1 Inside run_WADERs

option	effect
--gain-correction / pooled	one curve per module from all its images → root_gaincorr/
--gain-correction per-image	each image's own curve; a CCD whose comb is unusable gets the module curve (noted in the summary) → root_gaincorr_per_image/
--gain-correction both	both, plus gain_vs_charge/per_image_vs_pooled.csv (curves, energies, front-side Kα)
--gain-model	form of $g(n)$: monotone interpolation through the measured points (default), two lines with a fitted knot, or one line
--gain-combine	several images of a module: *coherent* (default; each image converted with its own curve, then the pixels pooled, so the result is independent of the image order) or *incoherent* (comb powers added)
--gain-anchor	*comb* (default; the comb's $g(n)$ throughout, which removes the image-to-image scatter of the fitted gains) or *fit* (each image's fitted gain kept at low charge)

The plots in gain_vs_charge/ are <name>_gain_vs_charge.png (local gain against charge) and <name>_nonlinearity.png ($A(n)/g(0) - n$ with its quadratic fit). CCDs with an untrustworthy curve (measured to $< 400\,\mathrm{e}^-$, or rising) are copied unchanged.

4.2 On its own

```
python src/gain_vs_charge.py measure img1.fz img2.fz ... -o OUTDIR [--name NAME] [--ext 1 3 4]
        [--model interpolated|broken|linear] [--combine coherent|incoherent] [--extrap-factor 3]
python src/gain_vs_charge.py apply OUTDIR/NAME_gain_vs_charge.json root/panaSKImg_clustersRec_*
    _ext*.root
        -o root_gaincorr [--anchor comb|fit]
```

measure writes <name>_gain_vs_charge.json (one model per extension) and the plots. apply rewrites pixels_E and Energy of each cluster file and copies info and process_config. Method note: docs/gain_correction_method/gain_correction_method.tex. Effect on the data: docs/gain_correction_report.pdf and docs/gain_correction_Am241.pdf.

5 Fe-55 composite analysis: --composite

After the cluster files are written, MakeComposite_Fe55_Clusters_Extensions.cc runs with ROOT, once per image type and module. It produces the front/back composite clusters, the Fe-55 Kα/Kβ fits and the CTI maps.

- **Input files**: root_gaincorr/ if --gain-correction is on, else root/. With --gain-correction both, root_gaincorr_per_image/ is also used, giving composite_per_image/. With --clusterer agmx, the A_GMX files are used instead; with both, they are used in addition and the results go to composite_agmx/.
- **Image type** N (the CCD geometry in the macro: 4, 5, 6, 7, 8, 11): from Image_<N> in the name, or --image-number.
- **Module name**: from --module ACM=NAME, else from a data folder named ..._DMa_DMb_DMc_... (ACM 110 → DMa, 109 → DMb, 106 → DMc), else ACM<id>.
- **Output**: composite/Fe_55_Analysis_Plots/<module>/Image_<N>_<module>_list/ (ext1..4/ per CCD, compare/ 2×2 overviews), composite/lists/, and logs/composite_Image_<N>_<module>.log.

6 A_GMX clustering

A_GMX (an adaptive regime switch plus Gaussian-mixture deblending) is the recommended alternative to panaSKImg's ClusterFinder; its algorithm is described in `docs/aGMX/aGMX_clustering.tex`. On 878 image-CCDs it gave:

- unbiased Fe-55 energies at every cluster width;
- about 2.7× better energy resolution;
- resolved close X-ray pairs;
- few-electron events without fakes;
- a correct energy scale on crowded CCDs.

It runs on the calibrated images that panaSKImg writes (`*_EXT<n>_waders.fits.bz2`) and produces cluster files **in the WADERs format**, so every tool that reads WADERs cluster files reads them as well.

6.1 Three ways to run it

```
# 1. in the pipeline (after panaSKImg and the gain correction)
run_WADERs images.txt --gain-correction --clusterer agmx --composite     # composite on A_GMX
run_WADERs images.txt --gain-correction --clusterer both --composite     # composite/ and
    composite_agmx/

# 2. on a finished run_WADERs folder
run_AGMX WADERs_analysis_<run> [-j 4] [--no-gaincorr] [--per-image] [--anchor comb|fit] [-o DIR]

# 3. the module directly
python src/agmx_root.py WADERs_analysis_<run> [-o OUTDIR] [--curve gain_vs_charge/ | curve.json]
    [-j N]
```

output folder	clusters of
`root_agmx/`	the images as calibrated by panaSKImg (fitted gain)
`root_agmx_gaincorr/`	the images converted with the module's $g(n)$ curve from `gain_vs_charge/` (same `--gain-anchor` as `root_gaincorr/`); written when the run has `gain_vs_charge/`
`root_agmx_gaincorr_per_image/`	the same with each image's own curve (`run_AGMX --per-image`, or `--gain-correction per-image/both`)

Each folder has `root_files.txt`. Speed: about 30–60 s per CCD (a crowded CCD takes longer). panaSKImg's ClusterFinder still runs, because its file for each CCD supplies the `info` and `process_config` trees and the seed settings. `root/` is never changed.

6.2 File format

The A_GMX clusters are turned into pysimdamicm `Cluster` objects, the class the ClusterFinder itself uses. Every branch is therefore computed by panaSKImg's own code, and the file is written with pysimdamicm's `OutputDataManager`:

- **Same as WADERs**: trees `clustersRec`, `info` and `process_config` (the last two copied from the CCD's panaSKImg file), and all WADERs branches: `Energy`, `PosX/Y`, `wSTD_*`, `fSTD_*`, `pixels_x/y/E`, `Npix`, `Qmax`, `has_seed`, the PCA and track variables, `closest_cls_*`, `Energy_PS/AVG`, `RUNID`, `ccd`, `nfile` and `readout_*`.
- **Same conventions**: `pixels_x/y` are 1-based; `pixels_E` is in keV with $e_{2\mathrm{eV}}$ from `process_config`, and `Energy` = $\sum$`pixels_E`. A pixel shared by two clusters is split between them, so the sum is the A_GMX energy.

- **Extra branches** (one value per cluster):

`agmx_cls`	class: 1 x-ray, 2 x-ray diffuse, 3 low-E, 4 muon, 5 worm, 6 alpha, 7 big deposit, 8/9 row/column artefact, 10 extended
`agmx_blend`	fraction of the energy shared with neighbours (deblending)
`agmx_ped`	local pedestal: 25th percentile of the charge 5–8 px around the cluster [e^-/pix]
`agmx_peakfrac`	largest Gaussian component / cluster
`agmx_chi2`	χ^2 per pixel of the cluster's own model
`agmx_sig`	significance of the cluster
`agmx_regime`	0 sparse, 1 crowded, 2 unmodelled (hysteresis)

The file also contains a `TNamed agmx_config` with the version, the parameters and the codes, as JSON.

Recommended selection for X-ray lines

`agmx_blend < 0.02` and `agmx_ped < 10`, plus the analysis' usual σ_{xy} cut (e.g. < 0.7 for the Am-241 lines, < 1 for front-side Fe-55). The `ped` cut removes clusters on a charge sheet. This is what made the high-side tail of the 59.5 keV line on the crowded CCD (docs/aGMX, section "Contamination").

Checked against WADERs

`clustering_tests/agmx_root_check.py PANASKIMG_ROOT WADERS_FITS EXT` feeds the clusters of a real panaSKImg file through the writer and compares every branch. On EXT1, EXT3 and EXT4 of an Am-241 image (1652, 7686 and 5452 clusters), every branch is identical except in numerically degenerate cases, where panaSKImg's own result depends on the last bits of its inputs:

- `fSTD` of profiles 1–3 bins wide;
- the sign of PCA eigenvectors (θ versus $\theta \pm \pi$);
- 1–2 pixel PCA;
- `closest_cls_*` when the nearest cluster lies in the same row or column. panaSKImg's "exclude itself" test is `PosX != ... and PosY != ...`, and the writer uses the same rule.

6.3 From Python

```
import sys; sys.path.insert(0, "comm_scripts/clustering_tests")
import adaptive as AD
clusters = AD.adaptive_cluster(Q, valid, engine="gmx")     # Q: image in electrons, valid: bool
    mask
good = [c for c in clusters if c["blend"] < 0.02 and c.get("ped", 0) < 10 and not c.get("edge")]
# each cluster: E [keV], y, x, sxy [px], cls, blend, ped, peakfrac, chi2, sig, edge, regime
# return_pixels=True also gives c["_pix"] = (rows, cols, electrons), summing exactly to E
```

`src/agmx_root.py` exposes the steps separately: `run(run_dir, outdir, curve, anchor, jobs)`, `process(...)` for one CCD, `build_clusters(...)`, `write_root(...)` and `read_reference(...)`.

7 The pipeline modules on their own

7.1 `autoconfig_panaSKImg.py`: a panaSKImg JSON from a raw image

```
python src/autoconfig_panaSKImg.py image.fz -o image.json [options]
python src/autoconfig_panaSKImg.py image.fz -o OUTDIR --per-ext      # one JSON per CCD + run_<
    image>.sh
```

It measures geometry, gain, noise, dark current, dead channels, bad columns and the pedestal method from the raw pixels. It changes only these settings in the reference JSON, which is built in (`-r my.json` uses your own). The same options can be passed through `run_WADERs --autoconfig-args "..."`.

Table 4: autoconfig options (also usable inside `--autoconfig-args`).

option	meaning (default)
`--gain G`	gain prior [ADU/e^-] if peaks are not resolved; also per CCD `EXT1=1.32,EXT2=1.30`
`--gain-min`, `--gain-max`	gain search range (0.6, 3.0)
`--prior-tol`	allowed relative difference measured gain vs prior (0.25)
`--max-sig-over-gain`	noise/gain above which the peaks count as unresolved (0.3)
`--lam-max L`	rows with more dark current than L e^-/pix are left out of the fit (1.0)
`--min-rows N`	minimum number of rows in the fit (25)
`--peaks auto|all|N`	peaks to fit: `auto` = those that follow one Poisson distribution (default)
`--poisson-tol`, `--max-peaks`	tolerance (0.25) and upper limit (10) of `--peaks auto`
`--no-fallback`	no held-gain $0\,e^-/1\,e^-$ fit for unresolved CCDs
`--ccd-cols N`	prescan+active columns (default from the header)
`--dead-frac F`, `--keep-dead`	detection of channels without a CCD (2×10^{-4}), or keep them
`--cluster-threshold-e P S | auto`	ClusterFinder thresholds in electrons (pixel $> P$, seed $> S$), or `auto` per CCD from its dark charge (smallest k with $P(\text{Poisson}(\lambda) \geq k) \leq 10\,\%$, pixel $> k - 0.5$, seed $> 9.5\,e^-$). Default: the reference's 10σ / 30σ
`--cluster-all`	also cluster flooded/outlier CCDs
`--plot-peaks N`	DC-fit plot shown up to N e^- (5; needs patched pysimdamicm, see `docs/patches/`)
`--tune`, `--tune-dir`	choose the pedestal method/window with panaSKImg; keep those runs
`--per-ext`	`-o` is a folder: one JSON per CCD and a run script
`--panaskimg PATH`	panaSKImg used by `--tune` and the run script

In `--autoconfig-args`, do not pass `--peaks`, `--per-ext` or `--tune`; `run_WADERs` sets these itself.

7.2 `gain_dc_split.py`: gain from all peaks, dark current from two

```
python src/gain_dc_split.py image.fz -o OUTDIR [--per-ext] [--run-full OUT] [-j 4] [--tune]
        [--autoconfig-args "..."] [--panaskimg PATH]
```

This is step 1 of `run_WADERs`. It writes `gain/`, `dc/` and `final/` JSONs and a summary comparing the fits; `--run-full OUT` also runs the full panaSKImg sequence.

7.3 `fit_quality.py`: how good are the dark-current fits

```
python src/fit_quality.py PANASKIMG_OUTPUT_FOLDER [--ref FOLDER ...] [--csv out.csv]
```

It reports χ^2/ndf, the scaled errors, and the dark-current bias against the $0\,e^-/1\,e^-$ counts of the autoconfig `*_measured.json`.

7.4 `gain_vs_charge.py` and `agmx_root.py`

See Sections 4 and 6.

8 Utilities

script	what it does and how to run it
compare_runs.py	Are two run_WADERs outputs identical? It compares summary.txt, fit_quality.csv, the final JSONs, the $g(n)$ curves, every cluster energy in root*/ and the composite tables. PDFs and logs are not compared. Exit code 0 if identical. python utils/compare_runs.py OLD NEW [-q]
split_clustersRec_by_ext.py	Splits files made with --acm (all CCDs stacked, _ext1234.root) into one file per CCD. python utils/split_clustersRec_by_ext.py files... -o OUTDIR [--nrows 500] [--list L]
compare_dcfit_vs_skips.py	Tiles the per-CCD plots of runs with different skip numbers (analysis/<skips>/) into one PDF per CCD. python utils/compare_dcfit_vs_skips.py analysis/ -o OUT [--ncols 4] [--plot FitDarkCurrentProcess_DCfit]
skip_noise_L1.py	Decodes a level-1 image (every skip kept, *.RJ45ADCDATA<ch>) and plots noise against the number of skips, the noise spectrum and the start-of-pixel transient. --composites N also writes N partial averages. It needs src/. python utils/skip_noise_L1.py FILE -o OUT [--geometry RxCxS] [--skip-first 20] [--gain G] [--max-rows N]

9 Research code: clustering_tests/

This folder holds every clustering method compared with WADERs in the study of 2026-10-23 to 2026-10-25, together with the scripts that tested them. Apart from A_GMX, which the writer uses, none of it is part of the pipeline.

- **Paths**: the scripts read data from absolute paths under ~/Documents/.
- **Outputs**: written next to the scripts (git-ignored).
- **Saved results**: ~/Documents/comm_scripts_data/session_2026-10-23_clustering/clustering_2026-10-25/.
- **Python**: the damic environment.

9.1 Methods

module	method	entry point and idea
adaptive.py	**A_GMX**	adaptive_cluster(Q, valid, engine="gmx"): regions the model cannot describe go to hysteresis; everything else goes to GMX
gmx.py	GMX	general_cluster(Q, valid, bg): Gaussian-mixture EM deblending, ridge/shape merging, island BIC, quality flags
mhc.py	MHC	specialised finders (X-ray, track, streak, low-E, ...) with χ^2/BIC arbitration and classes
methods2.py	hybrid v1/v2	MFD+G pieces merged by B-region and saddle tests (mfdg_hybrid), track-first
agent_methods.py	MFD+G	background map, Poisson tails, matched-filter detection + disk + growth
cluster_methods.py	WADERs-like, B	hysteresis (wader), topological clustering in electrons (topo), injection (inject)
pws.py	PWS	persistence-simplified watershed (a control; not recommended)

9.2 Test and validation scripts

script(s)	test
`wide_test.py`, `wide_adaptive.py`, `wide_gmx.py`, `wide_adaptive_gmx.py` (+ `summ_wide.py`)	every method on all 878 calibrated image-CCDs: injection, real Kα, 59.5 keV
`wide_agmx_flags.py` (+ `summ_wide_flags.py`)	Fe-55 Kα with the quality-flag cuts; injection
`am241_lines_flags.py` (`... fit`)	every Am-241 line, old vs new A_GMX, with/without the flag cuts
`adaptive_validate.py` (+ `run_*.sh`)	tracks, few-electron, Fe-55 and crowded-CCD validation
`compare.py`, `mhc_fe55.py` (+ `summ_fe55.py`)	Fe-55 injection at 4 widths, pairs, real Kα
`am_test.py`	Am-241 lines, injected 17.75 / 59.5 keV, splitting of big events
`tracks_test.py`, `extscan.py`, `mhc_tracks.py`	injected muons, worms, blobs, alphas, hot columns, pairs
`lowE_test.py`, `mhc_lowE.py`	few-electron events: efficiency, exact charge, fakes
`am241_29img_recluster.py`, `am241_29img_fit.py`, `adaptive_*_am241.py`, `make_crowded_table.py`	crowded Am-241 CCD: the constrained Lepy fit for each clustering; occupancy and threshold scans
`agmx_root_check.py`	the WADERs-format check of `src/agmx_root.py` (Section 6)

9.3 Documentation builders

`make_agmx_figures.py`	`docs/aGMX/figures/fig00-13`: every A_GMX step on a real image
`make_agmx_perf.py`	`fig20-26` and `docs/aGMX/perf_tables.tex`
`make_agmx_contamination_figs.py`	`fig30-34`: the 59.5 keV tail and the quality flags
`make_overleaf_zip.py`	`docs/aGMX/aGMX_clustering_overleaf.zip`
`make_clustering_report.py` (+ `report_*.py`)	`docs/clustering_methods_report.pdf`

10 Analysis notes and reports

10.1 Am-241 lines, 10–30 keV (`docs/am241_note/`)

This note covers the gain correction per gain period, the cluster-width cut $\sigma_{xy} < 0.7$, the constrained fit of the Np L lines (Lepy et al. ratios, one energy scale k), and the two extra lines at 22.1 and 24.9 keV. Its input is two `run_WADERs` outputs of the same images (`no_gaincorr/` and `gaincorr/`). Defaults are `--run-dir ~/Documents/JH1_Testing/AMTEst/runs_29img` (or `$AM241_RUN_DIR`) and `--run-name AMtest_29`.

step	run	does
0	`run_WADERs list.txt -o no_gaincorr --composite` `run_WADERs list.txt -o gaincorr --composite --gain-correction`	the pipeline (∼35 min each for 29 images)
1	`correct_gain_per_period.py --list list.txt --periods 21-40 41-50`	$g(n)$ per gain period
2	`extract_clusters.py`	cluster files → one table per CCD
3	`fit_all_peaks.py [--no-grid]`	all fits; the band-cut grid (∼40 min)
4	`peak_table.py`	mean, σ, FWHM and counts of every peak
5	`identification_tests.py`	tests of the extra-line hypotheses
6	`make_note_figures.py`	every figure and table of the note
–	`gold_check.py`, `make_report.py`	gold L lines; the companion report

The LaTeX source is `note/main.tex`; the Overleaf upload is `docs/AMtest_Am241_10-30keV_note_overleaf.zip`.

10.2 Gain correction

- `docs/gain_correction_method/gain_correction_method.tex`: the method note. Its figures are made by `make_figures.py`, and `am241_forms.py` compares the three model forms on Am-241.
- `docs/report_scripts/make_gain_correction_report.py` → `docs/gain_correction_report.pdf` (Fe-55 across the production data). `make_gain_correction_Am241.py` → `docs/gain_correction_Am241.pdf`.

10.3 Clustering

- `docs/aGMX/aGMX_clustering.tex` (Overleaf zip next to it): A_GMX step by step, with equations, figures, performance against WADERs, and the contamination study with its quality flags.
- `docs/clustering_methods_report.pdf`: all methods and all tests.

10.4 Hand-over notes

`docs/PIPELINE_GUIDE.md` gives the physics and the code map; read it first. `CONTEXT_2026-10-16.md`, `CONTEXT_2026-10-24.md`, `HANDOFF_2026-10-23_clustering.md` and `PROGRESS_2026-10-23.md` record what was done and what is still open.

11 Tests and changing the code safely

```
cd comm_scripts
python -m unittest discover -s tests -v          # with the damic / panaSKImg python
```

`test_run_wader.py`	input handling of `run_WADERs` (lists, names, options)
`test_gain_vs_charge.py`	the $g(n)$ models and their inversion
`test_am241_model.py`	the constrained Am-241 fit on synthetic spectra
`test_cluster_thresholds.py`	the automatic cluster thresholds in electrons
`test_clustering_methods.py`	MFD+G, hybrid, MHC and the GMX quality flags on a simulated image
`test_agmx_root.py`	the A_GMX writer: file structure, panaSKImg conventions, flags (simulated CCD)

For a change to the pipeline itself, run `run_WADERs` on the same images with the old and the new code, into different `-o` folders, and compare them with `utils/compare_runs.py OLD NEW`. The pipeline is deterministic, so any difference comes from the change. A git worktree of the old commit is a convenient way to run the old code.

12 Troubleshooting

symptom	**cause / fix**
"already exists; use `--overwrite`"	the output folder is not empty: `--overwrite`, or another `-o`
"no conda environment with panaSKImg"	`--env NAME` / `$WADERS_ENV`, or activate the environment
a CCD is "not run (flooded/unresolved)"	the peaks could not be resolved; `--all-ext` forces it (fits not reliable)
very slow clustering	high dark current: `--autoconfig-args "--cluster-threshold-e 2 5"` (or `auto`)
MakeComposite: "not run: no Image_<N>"	give `--image-number N`
composite module called ACM<id>	give `--module <id>=<name>`
"no g(n) curve for this CCD" (A_GMX)	the CCD's curve was not trustworthy; use the `root_agmx/` file
`x_max_plot` not supported	unpatched pysimdamicm: the wider DC-fit plot is left out (`docs/patches/pysimdamicm_x_max_plot.patch`)
`_ext` in the output path (warning)	MakeComposite reads the extension from `_ext<N>`; avoid it in folder names